

DE51 – Sujet ShareX
HERVE Alan, YOVO Yawo, DONG Zehui

Livrable final contenant les livrables précédents
ainsi que la conclusion de cet audit.

1. Introduction	4
2. Présentation du logiciel audité : ShareX	4
3. Critères non-techniques	4
4. Critères Techniques.....	5
4.1 Analyse des critères techniques de ShareX	7
5. Analyse du forum officiel ShareX sur Discord.....	7
5.1 Les réponses aux questions des utilisateurs.....	8
5.2 Investigation de ShareX par le biais de Discord	9
6. Analyse du Compte Officiel ShareX sur X (anciennement appelé Twitter)	10
6.1 La fréquence de publication de postes.....	10
6.2 La réponse aux questions des utilisateurs	11
6.3 Le Taux des questions résolues dans un jours	11
7. Détails de l'analyse de nos 3 critères subjectifs	12
7.1 Analyse de la part de Alan Hervé	12
7.1.1 Interface utilisateur	12
7.1.2 Complexité de l'application pour les nouveaux utilisateurs	13
7.1.3 Facilité d'utilisation pour les nouveaux utilisateurs	13
7.2 Analyse de la part de Zehui DONG	13
7.2.1 Interface utilisateur	13
7.2.2 Complexité de l'application pour les nouveaux utilisateurs	13
7.2.3 Facilité d'utilisation pour les nouveaux utilisateurs	13
8. Synthèse d'évaluation (diagramme de radar, analyse SWOT)	14
8.1 Note de critères techniques :.....	14
8.2 Diagramme de Radar pour les critères techniques :	15
8.3 Analyse SWOT du logiciel : Forces, Faiblesses, Opportunités et Menaces.	15
9. Pistes d'améliorations pour l'application ShareX.....	17
9.1 La dette technique accumulés par ShareX	17
9.1.1 Scénario 1 : Se débarrasser entièrement de la dette technique	17
9.1.2 Scénario 2 : Se débarrasser des parties les plus gênantes de la dette technique	18
9.2 Les améliorations à apporter au code	19
9.3 Les améliorations à apporter à leur organisation.....	19
9.4 Les améliorations à apporter à l'interface.....	19

9.5 Les améliorations à apporter au site web	20
9.6 Prévenir les utilisateurs sur le risque de confidentialité de leurs données.....	20
9.6.1 Scénario 1 : Désactiver l'upload automatique.....	21
9.6.2 Scénario 2 : Expliquer les risques liés à l'upload automatique	21
10. Conclusion.....	21

1. Introduction

Dans le cadre de l'UE DE51 de la filière développement de la formation d'ingénierie sous statut étudiant à l'UTBM, nous avons acquis les compétences relatives à l'audit informatique. Au terme de l'UE, nous avons eu un projet d'audit du logiciel ShareX. Dans ce document, nous présenterons le résultat de notre audit.

Dans une première partie, nous présenterons les différents critères qui ont fait l'objet de notre étude. Ensuite, nous ferons une analyse de ces critères et évaluerons le logiciel. Dans la dernière partie, nous ferons des propositions en vue d'améliorer le logiciel.

En nous basant sur les sources qui serviront à notre audit, nous retenons plusieurs critères d'évaluation que regroupons en deux catégories : les **non-techniques** et les **techniques**.

2. Présentation du logiciel audité : ShareX

ShareX est un logiciel gratuit et open-source conçu pour Windows. Il s'agit avant tout d'un outil très complet pour la capture d'écran et l'enregistrement vidéo de l'écran. Plus qu'une simple application de capture, ShareX intègre de nombreuses fonctionnalités qui en font un utilitaire puissant :

- Capture d'écran avancée : Il permet de prendre des captures de différentes manières (une partie de l'écran, une fenêtre, tout l'écran, ou même des pages web entières avec défilement).
- Enregistrement vidéo et GIF : enregistrer des vidéos de ce qui se passe sur l'écran ou créer des GIFs animés.
- Édition intégrée : Après la capture, ShareX offre un éditeur d'image simple pour annoter, dessiner, ajouter du texte ou des formes (flèches, cadres) directement sur l'image.
- Partage facilité : Une de ses forces majeures est la possibilité de télécharger automatiquement les captures et enregistrements vers de nombreux services en ligne (comme Imgur, Google Drive, Dropbox, etc.) et de vous fournir directement le lien de partage.
- Utilitaires additionnels : Il inclut aussi des outils pratiques comme un sélecteur de couleurs, une règle d'écran, un générateur de codes QR, et même une reconnaissance optique de caractères (OCR) pour extraire du texte d'une image.

Le logiciel étant présenté, nous allons énumérer les parties suivantes les différents critères sur lesquels s'est basé notre audit.

3. Critères non-techniques

4. Critères Techniques

Il s’agit des critères techniques pour évaluer la qualité du logiciel et le travail des développeurs. Voici un tableau des critères techniques et leurs descriptions.

Critère	Description	Outil	Barème	Valeur mesurée	Note										
Sécurité	Nombre d'utilisation d'une faible cryptographie, de protocoles non avancées, failles de sécurité critiques, possibilité d'une attaque DOS	Sonarqube	<table><tr><td>0</td><td>5</td></tr><tr><td>< 5</td><td>4</td></tr><tr><td><10</td><td>3</td></tr><tr><td>< 20</td><td>2</td></tr><tr><td>> 20</td><td>1</td></tr></table>	0	5	< 5	4	<10	3	< 20	2	> 20	1	96	1
0	5														
< 5	4														
<10	3														
< 20	2														
> 20	1														
Fiabilité	Nombre de mauvaises pratiques qui peuvent causer des mauvais comportements lors de l'exécution (mauvaises comparaisons, deadlocks, etc)	Sonarqube	<table><tr><td>A</td><td>5</td></tr><tr><td>B</td><td>4</td></tr><tr><td>C</td><td>3</td></tr><tr><td>D</td><td>2</td></tr><tr><td>E</td><td>1</td></tr></table>	A	5	B	4	C	3	D	2	E	1	60 issues (note D)	2
A	5														
B	4														
C	3														
D	2														
E	1														
Maintenabilité	Respect des conventionsn de codage (nommages, complexité cognitive, encapsulation, etc)	Sonarqube	<table><tr><td>A</td><td>5</td></tr><tr><td>B</td><td>4</td></tr><tr><td>C</td><td>3</td></tr><tr><td>D</td><td>2</td></tr><tr><td>E</td><td>1</td></tr></table>	A	5	B	4	C	3	D	2	E	1	1,946 issues (note A)	5
A	5														
B	4														
C	3														
D	2														
E	1														
Duplication de code	Pourcentage de duplication de code	Sonarqube	<table><tr><td>< 1%</td><td>5</td></tr><tr><td>< 2%</td><td>4</td></tr><tr><td>< 5%</td><td>3</td></tr><tr><td><10%</td><td>2</td></tr><tr><td>>15%</td><td>1</td></tr></table>	< 1%	5	< 2%	4	< 5%	3	<10%	2	>15%	1	1,5 %	4
< 1%	5														
< 2%	4														
< 5%	3														
<10%	2														
>15%	1														
Couverture des tests	Pourcentage de code couvert par des tests	Sonarqube	<table><tr><td>>80%</td><td>4</td></tr><tr><td>>60%</td><td>3</td></tr><tr><td>>40%</td><td>2</td></tr><tr><td>>20%</td><td>1</td></tr><tr><td>0%</td><td>0</td></tr></table>	>80%	4	>60%	3	>40%	2	>20%	1	0%	0	0 %	0
>80%	4														
>60%	3														
>40%	2														
>20%	1														
0%	0														
Facilité d'évolution	Ce critère mesure à quel point le code est	NDepend	<table><tr><td><0.2</td><td>5</td></tr></table>	<0.2	5	0.94	1								
<0.2	5														

Commenté [YY1]: Moyenne d'Instabilité (I) Note /5
Interprétation I ≤ 0.2 5 Très stable – utilisé largement par d'autres, difficile à modifier 0.2 < I ≤ 0.4 4 Plutôt stable – dépendance modérée, maintenance acceptable 0.4 < I ≤ 0.6 3 Équilibré – bon compromis stabilité / flexibilité 0.6 < I ≤ 0.8 2 Plutôt instable – dépend de nombreux modules, fragile I > 0.8 1 Très instable – dépend fortement d'autres, peu utilisé, changement peu risqué

	facile à faire évoluer. Il se base sur l' instabilité de chaque composant, c'est-à-dire sa dépendance aux autres. Plus un module est instable (fortement dépendant), plus il est flexible, mais aussi fragile. Inversement, un module trop stable est difficile à modifier. L'équilibre est donc recherché. Une instabilité trop élevée indique une architecture peu robuste.		<table><tr><td>< 0.4</td><td>4</td></tr><tr><td>< 0.6</td><td>3</td></tr><tr><td><0.8</td><td>2</td></tr><tr><td>> 0.8</td><td>1</td></tr></table>	< 0.4	4	< 0.6	3	<0.8	2	> 0.8	1	(Moyenne des instabilités de chaque module)			
< 0.4	4														
< 0.6	3														
<0.8	2														
> 0.8	1														
Qualité d'abstraction	Évalue la capacité du projet à séparer la définition (interfaces, classes abstraites) de l'implémentation concrète. Une bonne abstraction rend le code plus extensible, testable et maintenable. Une abstraction quasi-nulle indique une dépendance forte à l'implémentation concrète, ce qui rend les évolutions plus coûteuses. Cette métrique repose sur le ratio entre types abstraits et types concrets dans chaque module.	NDepend	<table><tr><td>0.3 - 0.6</td><td>5</td></tr><tr><td>0.2 - 0.3</td><td>4</td></tr><tr><td>0.1 - 0.2</td><td>3</td></tr><tr><td><0.1</td><td>2</td></tr><tr><td>0</td><td>1</td></tr></table>	0.3 - 0.6	5	0.2 - 0.3	4	0.1 - 0.2	3	<0.1	2	0	1	0.035 (moyenne de Abstractness des modules)	2
0.3 - 0.6	5														
0.2 - 0.3	4														
0.1 - 0.2	3														
<0.1	2														
0	1														
Distance architecturale	Indique si les modules sont bien positionnés dans l'architecture globale, selon le principe de la "main	NDepend	<table><tr><td>< 0.1</td><td>5</td></tr><tr><td>< 0.3</td><td>4</td></tr></table>	< 0.1	5	< 0.3	4	0.037 (moyenne de Abstractness des modules)	2						
< 0.1	5														
< 0.3	4														

	sequence" de Robert C. Martin. Un bon module doit être soit stable et concret, soit instable et abstrait. Ce critère mesure l'écart entre l'abstraction et l'instabilité idéales. Plus la distance est faible, meilleure est la qualité architecturale.		<	3		
			0.5			
			<	2		
			0.7			
			>	1		
			0.7			

Commenté [YY2]: Distance D Note Raison ≤ 0.1 5
 Presque parfait : proche de la ligne idéale 0.1 – 0.3 4
 Légère déviation acceptable 0.3 – 0.5 3 Distance
 modérée, à surveiller 0.5 – 0.7 2 Assez loin : design
 potentiellement problématique > 0.7 1 Très éloigné :
 violation des principes d'architecture recommandés

4.1 Analyse des critères techniques de ShareX

Les premiers critères ont été évalués par SonarQube. SonarQube est une plateforme d'analyse statique qui inspecte le code source à la recherche de bugs, vulnérabilités, dettes techniques, duplications, et problèmes de style. Elle attribue des scores quantitatifs par métrique, ce qui permet une évaluation rigoureuse. Vous pouvez retrouver l'analyse sur ce serveur :

http://4.206.215.231:9000/dashboard?id=DarkBrain-LP_ShareX_698bd895-f44a-440f-8167-2707c805eb1a&codeScope=overall (username: admin, password: Test1234567)

Pour 3 derniers critères, nous avons extrait des métriques par projet (ou assembly) à partir du rapport généré par NDepend après analyse du code source de ShareX. Ces métriques sont accessibles dans les rapports HTML, les graphiques de dépendances, ou les exports CSV/XLS via les requêtes CQLinq de NDepend.

Le logiciel étant présenté, nous allons énumérer les parties suivantes les différents critères sur lesquels s'est basé notre audit.

5. Analyse du forum officiel ShareX sur Discord

Le forum officiel de ShareX sur Discord est un point de regroupement majeur pour les utilisateurs de Sharex.

Il est regroupé en salons textuels pour :

- Poser des questions, demander conseil
- Discuter de ShareX en général
- Discuter de l'aspect programmation de ShareX
- D'autres salons hors-sujets

Certains rôles sont attribués au membre du server tels que :

- Administrateur
- Habitué
- Premium

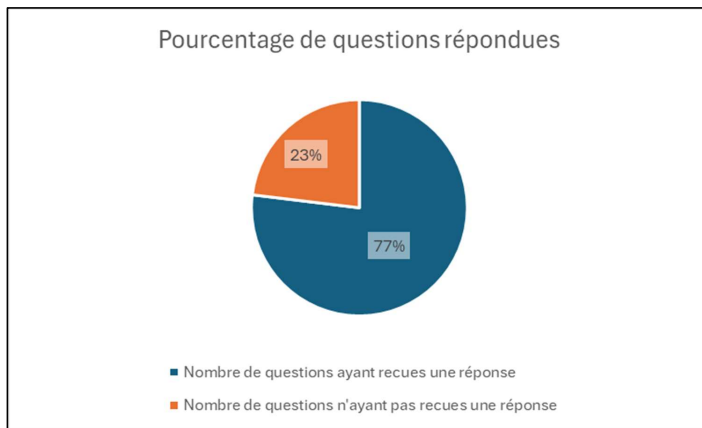
Les comptes utilisateurs (y compris ceux ayant le rôle Administrateur) sont à l'évidence des comptes personnels.

Point négatif pour le forum, les utilisateurs ne sont pas capables de se servir de la fonction "Thread" de discord pour améliorer la visibilité du site. Les questions et réponses sont donc dans le désordre avec à certains moments une question posée, puis une réponse à une question précédente.

5.1 Les réponses aux questions des utilisateurs

Dans l'intervalle de temps sélectionnée, la plupart des questions ont été posées par des utilisateurs ne possédant aucun rôle (des utilisateurs non-"Habitué" donc). La plupart des réponses aux questions venaient de la part d'utilisateurs avec le rôle Administrateur ou Habitué (c'est à dire des personnes activement impliquées dans ShareX ou des personnes utilisant l'outil de façon fréquente).

Peu de questions sont laissées sans réponses, et encore moins de questions sont totalement ignorées :



Le délai de réponse aux questions est aussi très court, parmi les questions posées seules une minime partie n'a pas reçu de réponse **définitive** le jour-même (il s'agissait de questions plus complexes ayant nécessité plus de temps pour arriver à une réponse ou une solution).



5.2 Investigation de ShareX par le biais de Discord

Pendant notre analyse des différentes questions et réponses, nous avons décidé d'essayer de demander conseils aux utilisateurs de ShareX sur la partie technique du programme :

- Demandes de renseignements sur une documentation de la partie code de ShareX (question posée le 22/04/2025 à 20h37)
 - o Réponse : "Not Really" (traduction : pas vraiment) nous en concluons donc qu'une documentation du code n'a jamais été proprement établie
- Demandes de renseignements sur la façon dont les personnes peuvent collaborer au code (comment les codes sont approuvés, si une potentielle convention de codage est établie et est nécessaire de suivre pour pouvoir collaborer au projet). Nous avons posé la question en demandant conseil pour potentiellement collaborer à notre premier projet open source (question posée le 22/04/2025 à 21h24)
 - o Réponse : *"Gonna be honest, ShareX might not be a good first, it's old, quite complex, and nowhere near as well documented as some other projects, combine that with Jaex being strict with PRs, it's going to be a challenge, so I would honestly recommend having a look around for simpler projects that can help you work your way up (I don't have any recommendations before you ask)"*
 - o Traduction : ShareX n'est pas un bon projet pour débiter en Open Source. C'est un projet ancien, complexe et très peu documenté par rapport à d'autres projets. De plus Jaex (le créateur de ShareX) est strict sur la façon dont le code est implémenté. Je recommanderais plutôt de chercher d'autres projets plus simples pour débiter.
 - o Ce message nous a aussi permis de découvrir un grand nombre de critiques à l'égard de Jaex. Nous ne développerons pas plus ce sujet, mais il semble que la façon de Jaex de

superviser le projet est critiquée par la communauté et les collaborateurs (les messages critiquant Jaex ont été supprimés une dizaine de minute après avoir été postés, ceci pourrait être signe d'une potentielle censure ou une tentative de ne pas se mettre Jaex à dos).

- Après cette réponse nous avons cherché à savoir s'il existait des "guidelines" pour les collaborateurs. Cette question nous a par ailleurs permit d'en apprendre plus sur la documentation de ShareX (question posée le 22/04/2025 à 21h50).
 - o Réponse : "Not that I know of. If something you're looking for isn't in the readme, info files or on the website it probably doesn't exist"
 - o Traduction : Pas à ma connaissance, si tu recherches une ressource non présente sur les fichiers readme et info ou sur le site, c'est probablement qu'elle n'existe pas
 - o Nous devons donc en conclure qu'il n'existe pas de guidelines pour les nouveaux contributeurs et que peu de ressources documentaires existent, y compris dans des fichiers non visibles du public.

6. Analyse du Compte Officiel ShareX sur X (anciennement appelé Twitter)

Le compte officiel de ShareX sur X (ancien Twitter) est un compte officiel pour publier les actualités sous forme des "Postes" et répondre aux questions des utilisateurs si besoin.

Il est maintenu pour :

- Publier les dernières mises à jour du logiciel.
- Correction des vulnérabilités et rapports sur les correctifs
- L'annonce sur les grands événements en ligne
- Répondre aux questions des utilisateurs si besoin

Point négatif pour le compte X (Twitter), manque d'interactivité : les utilisateurs sont souvent plus motivés par les forums tels que Discord. Ce n'est pas un forum assez adapté pour un logiciel technique.

6.1 La fréquence de publication de postes.

Période d'analyse : 23/04/2024 - 22/04/2025

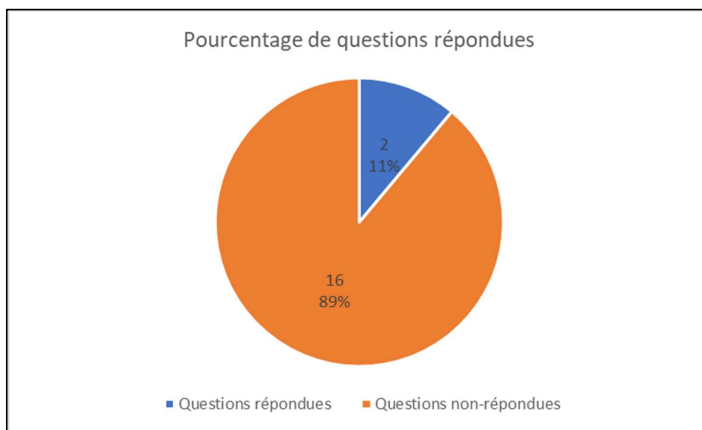
Nombre de postes publiés : 10

La fréquence de publication = 10 postes / 12 mois = Un poste tous les 1 – 3 mois

Nous pouvons constater que la fréquence de publication n'est pas très élevée, ce qui indique que le compte n'est pas très actif.

6.2 La réponse aux questions des utilisateurs

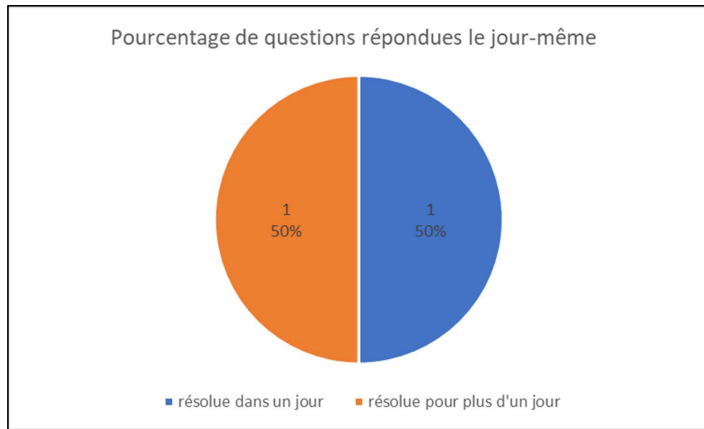
- Nombre de questions répondues : 2
- Nombre de questions non-répondues : 16
- Pourcentage de questions répondues : 11%



D'après le diagramme, le Taux de réponse des questions est $2/18=11.1\%$, ce qui signifie que le compte officiel X (Twitter) n'est pas bien géré et ne s'attache pas à répondre aux questions des utilisateurs.

6.3 Le Taux des questions résolues dans un jours

- Nombre de questions répondues le jour-même : 1
- Nombre de questions répondues pour plus d'un jour : 1
- Pourcentage de questions répondues le jour-même : $1/2=50\%$



Vue que la taille d'échantillon n'est pas assez grande, nous ne pouvons pas déduire de nouvelles conclusions. L'idée est la même que comme au précédent : le compte officiel X (Twitter) n'est pas assez actif par rapport au forum Discord. Une meilleure maintenance et des mises à jour plus fréquentes sont nécessaires pour mieux fonctionner.

7. Détails de l'analyse de nos 3 critères subjectifs

7.1 Analyse de la part de Alan Hervé

7.1.1 Interface utilisateur

L'interface utilisateur est simple et intuitive. Cependant, elle repose trop (à mon opinion) sur l'utilisation de drop-downs. J'aurais apprécié avoir l'option de pouvoir ouvrir des pages entières pour choisir certaines de mes options. Autre point pénalisant l'interface est son apparence : les couleurs sont bien choisies mais la police d'écriture ainsi que la qualité d'image des différentes icônes donnent l'impression que ShareX est un outil ayant été développé en 2007 sans recevoir de mises à jour graphiques depuis.

7.1.2 Complexité de l'application pour les nouveaux utilisateurs

ShareX est principalement conçu pour être utilisé par des personnes habituées à l'utilisation de différents logiciels informatiques. La page d'ouverture de l'application ne liste que des raccourcis claviers pour des captures d'écran spécifique et un nouvel utilisateur ne se voit pas proposer de suivre un tutoriel ou lire une présentation de l'application (à l'inverse d'autres logiciels comme Adobe Acrobat Reader). Pour cette raison j'ai choisi d'enlever 1 point à leur note.

Cependant, pour une personne habituée à utiliser différents logiciels et à "toucher à tout" pour découvrir le fonctionnement de l'application ainsi que ses fonctionnalités offertes, la plupart des fonctionnalités sont accessibles et maîtrisables dans les 10 premières minutes d'utilisation.

7.1.3 Facilité d'utilisation pour les nouveaux utilisateurs

L'application est facile à prendre en main pour les personnes habituées aux outils informatiques. Cependant, des fonctionnalités avancées telles que les "actions après la capture" (pouvoir passer l'image capturée à travers une application tierce ou son propre code) ne sont pas expliquées suffisamment clairement. Il existe une explication, mais il m'a fallu chercher dans l'application pour la trouver et cette fonctionnalité repose sur l'utilisation de lignes de commande. Pour cela j'ai décidé de déduire leur note de 1 point. Je n'ai pas relevé de problèmes supplémentaires.

7.2 Analyse de la part de Zehui DONG

7.2.1 Interface utilisateur

Je vote pour cette interface, qui est simple et assez intuitive. Les utilisateurs peuvent trouver facilement les paramètres et outils dans ce logiciel. Une remarque : l'interface peut être plus conviviale pour que les utilisateurs puissent faire plaisir avec leur travail.

7.2.2 Complexité de l'application pour les nouveaux utilisateurs

En tant qu'une application de captures d'écrans, l'application lui-même est très facile à installer et à la prise à main. Les utilisateurs peuvent facilement maîtriser les utilisations basiques. L'application demande aux utilisateurs de choisir leur emplacement des captures d'écran personnalisée, ce qui facilite l'utilisation. Une petite remarque sur le respect de la vie privée des utilisateurs : les paramètres de téléchargement automatique dans la zone avancée est « activé » par défaut. L'application va peut-être télécharger les captures d'écrans et avoir l'accès des captures d'écran de l'utilisateur si les paramètres ne sont pas bien gérés.

7.2.3 Facilité d'utilisation pour les nouveaux utilisateurs

Je ne dirais pas qu'il est assez intuitif, surtout si quelqu'un l'utilise pour la première fois. Beaucoup d'utilisateurs font leur première capture d'écran, et ont été surpris que toute l'interface de Windows se soit soudainement « figée ».

8. Synthèse d'évaluation (diagramme de radar, analyse SWOT)

Pour mieux évaluer la qualité du logiciel ShareX, nous avons procédé à une synthèse des évaluations selon nos critères. Nous vous rappelons les critères techniques de ShareX qui sont quantifiées par leurs notes. Nous avons également réalisé une analyse SWOT afin de mieux comprendre la performance de ShareX en quatre dimensions (Forces, faiblesses, opportunités, menaces).

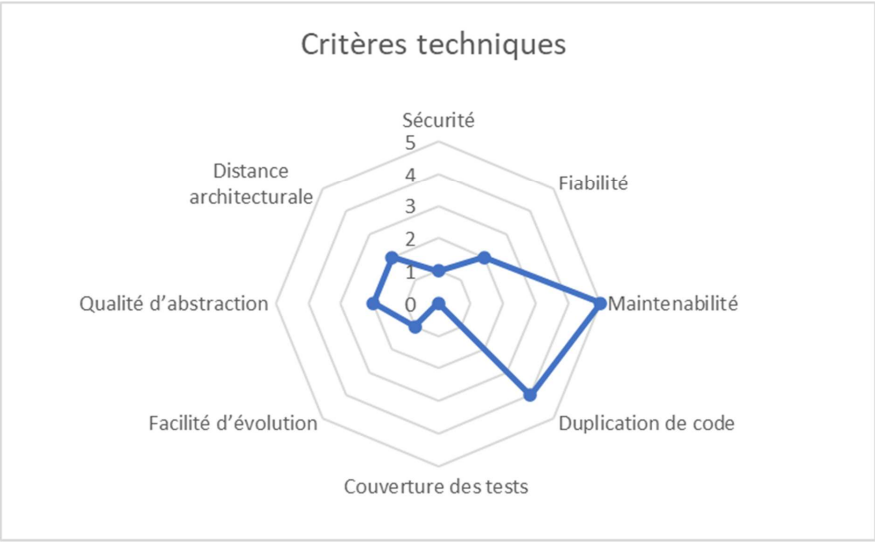
8.1 Note de critères techniques :

Nous vous présentons l'ensemble des critères techniques ainsi que leurs notes correspondantes dans ce tableau ci-dessous. En parallèle, un diagramme de radar a été réalisé afin d'illustrer ces barèmes de manière plus visuelle et intuitive.

Critère	Note
Sécurité	1
Fiabilité	2
Maintenabilité	5

Duplication de code	4
Couverture des tests	0
Facilité d'évolution	1
Qualité d'abstraction	2
Distance architecturale	2

8.2 Diagramme de Radar pour les critères techniques :



Ce diagramme de radar mesure tous les critères techniques évalués par SonarQube. Nous pouvons trouver que les notes pour la sécurité, la fiabilité, la couverture des tests, la facilité d'évolution, la qualité d'abstraction et la distance architecturale sont très basses.

En revanche, sa maintenabilité est excellente, ce qui facilite la continuation sur l'existant et l'ajout de nouvelles fonctionnalités. De plus, la duplication de code est bien gérée, c'est à dire qu'il n'existe pas beaucoup de répétition du code.

En conclusion, la performance générale du logiciel n'est pas assez satisfaisante. Ce logiciel présente de beaucoup d'issues qui doivent être résolues, celles-ci peuvent potentiellement causer une dette technique accumulé. Nous allons l'analyser dans la partie "Pistes d'améliorations pour l'application ShareX".

8.3 Analyse SWOT du logiciel : Forces, Faiblesses, Opportunités et Menaces.

SWOT	Aspects internes	Aspects externes
------	------------------	------------------

Forces	- Maintenabilité élevée (5/5) : code relativement bien structuré pour de l'open-source, facilite les contributions et la correction de bugs. - Faible duplication de code (4/5) : Moindre la divergence fonctionnelle, facile à reconstituer.	- Libre et gratuit : faible barrière à l'adoption, forte viralité. - Multi-services : captures, enregistrement vidéo, upload automatique sur divers serveurs/clouds. - Communauté active : nombreux plugins, scripts et tutoriels.
Faiblesses	- Couverture de tests très faible (0/5) : La moindre régression passe inaperçue. - Qualité d'abstraction et distances architecturales basses (2/5) : logique métier et UI trop entremêlées. - Faible Fiabilité (2/5) : Nombre de mauvaises pratiques élevée, qui peuvent causer des mauvais comportements lors de l'exécution	- Manque de support multiplateforme : Windows uniquement. - UX parfois austère : Prise en main initiale complexe pour l'utilisateur non-technique.
Opportunités	- Renforcer la QA : Ajouter un pipeline de tests automatisés (unitaires + UI) pour stabiliser et faciliter les refactors (tirer parti de la maintenabilité existante).	- Intégration IA/OCR : Reconnaissance de texte sur captures. - Extensions vers macOS/Linux : Élargir la base d'utilisateurs. - API tierces : S'intégrer à des outils collaboratifs (Teams, Slack...).
Menaces	<i>(Points faibles internes dégradés)</i>	<i>(Réticences ou concurrents externes)</i>
	- Dette technique accumulée : Sans refactoring, le projet peut devenir ingérable malgré sa maintenabilité actuelle. - Sécurité sous-évaluée (1/5) : Vulnérabilités potentielles non détectées faute de testes. - Téléchargement involontaire de données sensibles vers des services d'hébergement en ligne (Imgur par défaut) - Problèmes de confidentialité avec les services tiers	- Outils natifs et concurrents commerciaux : Snagit, Greenshot, les outils intégrés Windows/Mac. - Politiques de sécurité IT : entreprise bloquant les uploads tiers. - Désintérêt du mainteneur principal : Projet open-source très dépendant d'un ou deux contributeurs clés.

Ce formulaire au-dessus représente les résultats de l'analyse SWOT (Forces, Faiblesses, Opportunités, Menaces) du logiciel ShareX. Nous avons intégré l'ensemble des résultats de l'analyse et des critères d'évaluations, ainsi que les comportements du logiciel ShareX et ceux-ci de ses concurrentes. Il vise à présenter le résultat de notre analyse de manière plus intuitive et à proposer les axes d'amélioration pour l'application.

9. Pistes d'améliorations pour l'application ShareX

9.1 La dette technique accumulés par ShareX

La dette technique actuelle de ShareX est estimée par SonarQube comme étant de 43 jours. Ceci équivaut à 1032 heures de dette technique.

9.1.1 Scénario 1 : *Se débarrasser entièrement de la dette technique*

Pour rappel, ShareX est un projet open source qui ne reçoit pour seul financement que les éventuelles donations de ses utilisateurs.¹

Cependant, comme il s'agit d'un projet open source collaboratif, une partie des collaborateurs travaille sur la base du volontariat.

Dans cette situation nous allons considérer que tous les collaborateurs sont des employés rémunérés au même tarif qu'un ingénieur informaticien débutant :

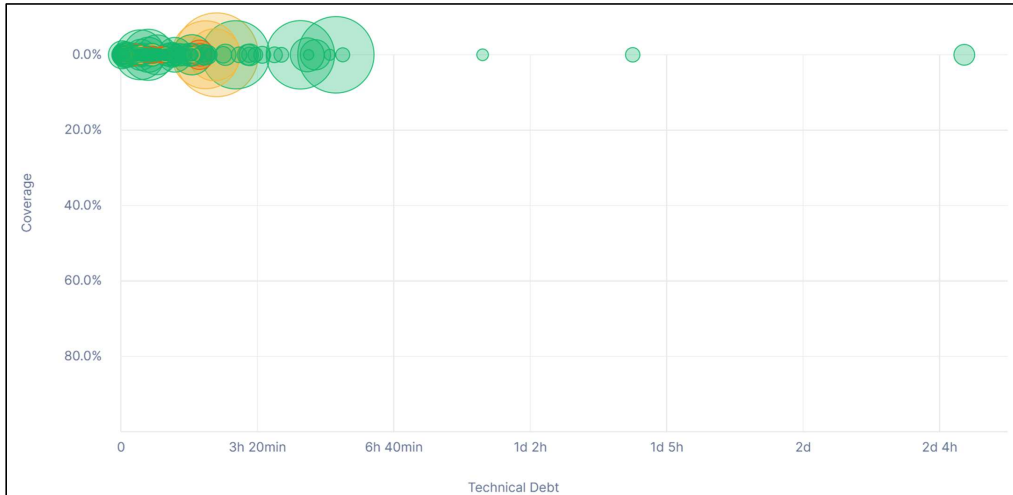
- Selon l'UTBM la rémunération actuelle pour un ingénieur débutant est de 37.800€ brut annuel sans prime². Ce qui prenant en compte les congés et vacances correspond à un salaire brut d'environ 3.150€ par mois ou environ 22€ de l'heure.
- Se débarrasser entièrement de la dette technique coûterait donc 22 704€ en salaire employé.

¹ SHAREX. Donate – ShareX [en ligne]. Disponible sur : <https://getsharex.com/donate> (Consulté le 01/06/2025).

² UTBM. Synth-E2023-5FISE.pdf [en ligne]. Dispnable sur : <https://www.utbm.fr/wp-content/uploads/2024/01/Synth-E2023-5FISE.pdf> (Consulté le 01/06/2025).

Cette solution est non seulement onéreuse mais devrait enlever un ingénieur qui pourrait participer à développer l'application et rajouter de nouvelles fonctionnalités.

Nous allons donc proposer un autre scénario.



9.1.2 Scénario 2 : Se débarrasser des parties les plus gênantes de la dette technique

Une solution plus réaliste pour l'équipe de ShareX serait de s'intéresser uniquement aux problèmes de niveau "haut" et "bloquant".

- Il existe 9 problèmes bloquants, SonarQube estime le temps nécessaire pour régler ces problèmes à 2h40min.
- Il existe 223 problèmes de niveau "high", SonarQube estime le temps nécessaire pour régler ces problèmes à 3j6h.

Cela correspond donc à un temps de 3 jours, 8 heures et 40 minutes. Soit, en heures, 80 heures et 40 minutes.

Comme mentionné précédemment nous considérerons que le salaire d'un collaborateur de ShareX serait de 22€ par heure. Ce qui nous amène à un total de 1 774,66 € pour régler ces problèmes.

Cette solution permet de s'assurer que les morceaux de codes pouvant amener à un "crash" de l'application soient pour la plupart corrigés et est beaucoup moins onéreuse que la solution précédente.

Nous tenons à soulever le fait qu'en implémentant cette stratégie il restera toujours de nombreux morceaux de codes problématiques au sein de l'application. Cependant, ces morceaux de codes problématiques étant moins importants (moins d'impact sur le fonctionnement de l'application) il pourra être possible de s'en occuper un peu chaque mois au fur et à mesure jusqu'à leur disparition.

Le plus important reste bien sûr de s'assurer de ne pas rajouter de nouveaux morceaux de codes problématiques au code l'application.

9.2 Les améliorations à apporter au code

Pour nous familiariser avec ShareX, nous avons parcouru son code et avons relevé plusieurs problèmes, notamment sur la difficulté d'essayer de collaborer à l'application pour des nouveaux contributeurs.

Voici les solutions que nous proposons pour régler ces problèmes :

- Plus de commentaires pour aider les nouveaux collaborateurs à s'intégrer au projet plus rapidement.
- Etablir un guide officiel des conventions de codage ainsi que de l'architecture utilisés dans l'application.
- Implémenter des tests unitaires : dans l'état actuel des choses des changements importés pourraient avoir des impacts imprévus graves.

9.3 Les améliorations à apporter à leur organisation

En utilisant SonarQube, nous nous sommes rendu compte qu'il existait beaucoup de code smells au sein du code de ShareX. Un de ces code smells est même une erreur bloquante qui pourrait amener l'application à "crash" (il s'agit de l'utilisation d'un simple "|" au lieu de "||" dans une condition if).

Ceci nous permet de voir que le code présent dans les push n'est pas forcément entièrement analysé ou lu par un membre de l'équipe avant que la merge request ne soit acceptée.

Il serait bénéficiaire pour ShareX de mettre en place un logiciel afin de surveiller la qualité de leur code (pour information, nous avons utilisé docker pour mettre en place un server SonarQube gratuitement, il est donc possible pour les responsables d'un logiciel open source d'en faire autant).

9.4 Les améliorations à apporter à l'interface

L'interface actuelle de ShareX n'est pas spécialement intuitive ou des plus élégantes. Pendant nos essais du logiciel nous avons remarqué plusieurs problèmes sur le nombre de clics à opérer avant d'atteindre une fonction désirée où l'apparence globale du logiciel.

Voici nos propositions d'amélioration :

- Améliorer les icônes pour donner un côté un peu plus moderne.
- Rajouter un bouton help qui mènerait vers des didacticiels.
- Trop de dropdown dans des dropdown. Une solution pourrait être d'offrir la possibilité d'ouvrir une page dédiée à une fonctionnalité au lieu d'avoir à ouvrir tous ces dropdowns.

9.5 Les améliorations à apporter au site web

Pendant notre phase de recherche sur le logiciel nous avons bien sûr du nous rendre sur le site web officiel de celui-ci. Nous avons donc eu l'occasion de repérer deux problèmes conséquents :

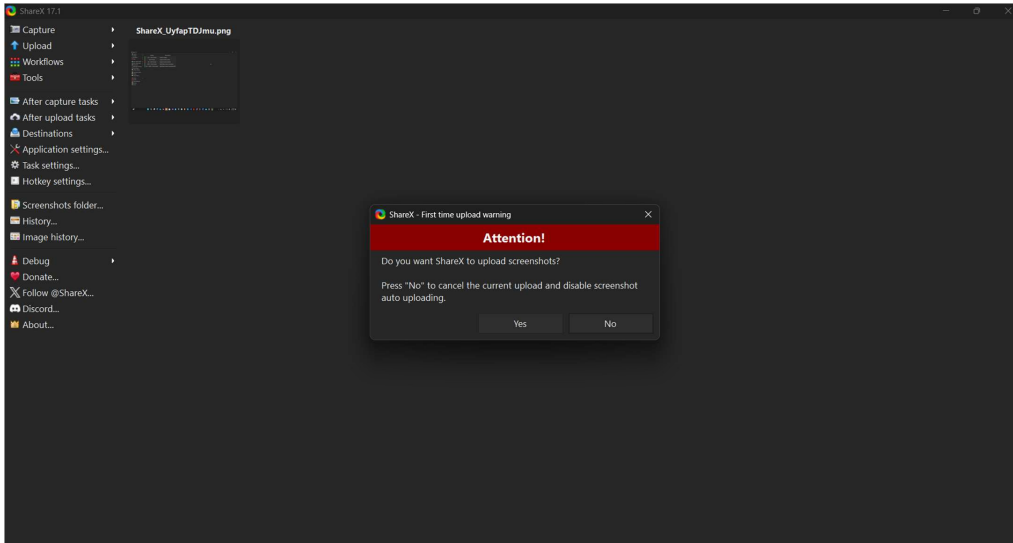
- Une page du site consiste juste de différentes captures d'écran sans préciser à quoi ces captures correspondent ou comment elles ont été obtenues.
- Le site ne vend pas très bien l'application, il ne met pas vraiment en valeur les différentes fonctionnalités par d'autres méthodes que tout simplement les énoncer une par une.

Nous sommes donc arrivés à la proposition d'amélioration suivante :

- Au lieu de simplement énoncer les fonctionnalités, chaque fonctionnalité devrait être accompagnée d'une capture d'écran montrant le résultat obtenu. De plus, il devrait être possible de cliquer sur ces fonctionnalités pour obtenir une sorte de guide qui expliquerait comment s'en servir. Ce guide remplacerait la page ne consistant que de captures d'écran que nous avons mentionnée précédemment, faciliterait la prise en main du logiciel et rajouterai un côté plus "visuellement attractif" au site.

9.6 Prévenir les utilisateurs sur le risque de confidentialité de leurs données

L'upload d'image sur imgur est configuré par défaut. A la première capture d'écran, une confirmation est demandée à l'utilisateur sans pour autant donner de détails sur cet upload.



9.6.1 Scénario 1 : Désactiver l'upload automatique

Dans les paramètres, on peut activer ou désactiver l'upload automatique de fichier. A l'installation, l'upload automatique est activé et configuré pour envoyer les images de manière anonyme sur Imgur. Il serait préférable de désactiver l'envoi de fichier par défaut.

9.6.2 Scénario 2 : Expliquer les risques liés à l'upload automatique

Une autre option serait de mettre en place un tour de découverte de l'application après l'installation qui inclurait les détails de l'envoi automatique sur Imgur. Ainsi, on s'assure que l'utilisateur serait bien conscient que ses images seraient publiquement disponibles sur Imgur quand il confirme l'upload automatique.

10. Conclusion

En conclusion, cet audit de ShareX révèle une performance mitigée, avec des points forts notables en tant qu'outil gratuit, open-source et polyvalent, doté d'une communauté active. Cependant, il met également en lumière des lacunes importantes. Du point de vue non-technique, ShareX offre une interface utilisateur globalement simple et intuitive pour les utilisateurs familiarisés avec les logiciels. Le forum Discord est un atout majeur, la plupart des questions recevant des réponses rapides, souvent le jour même, de la part des administrateurs ou des utilisateurs expérimentés. Néanmoins, l'absence de la fonction "Thread" sur Discord rend les discussions désorganisées. L'application présente une complexité initiale pour les nouveaux utilisateurs non-techniques, due à l'absence de tutoriels ou de présentations. L'interface visuelle est également jugée vieillotte. Le compte officiel ShareX sur X (anciennement Twitter) est peu actif, avec une faible fréquence de publication et un taux de réponse très bas aux questions des utilisateurs (11,1%), ce qui en fait un canal de support inefficace par rapport à Discord.

Sur le plan technique, ShareX excelle en maintenabilité (5/5) et présente une faible duplication de code (4/5), indiquant une structure de code relativement saine pour un projet open-source, ce qui facilite les contributions et les corrections de bugs. Cependant, l'audit révèle de sérieuses faiblesses techniques. Les scores sont très bas en sécurité (1/5), fiabilité (2/5), couverture des tests (0/5), facilité d'évolution (1/5), qualité d'abstraction (2/5) et distance architecturale (2/5). Le manque de couverture des tests (0%) est particulièrement préoccupant, augmentant le risque de régressions non détectées. La faible fiabilité indique de nombreuses mauvaises pratiques de codage pouvant entraîner des comportements indésirables. Une faible abstraction et une mauvaise distance architecturale suggèrent un couplage fort entre la logique métier et l'interface utilisateur, rendant le code moins robuste et plus difficile à modifier. La dette technique accumulée est estimée à 43 jours (1032 heures).

Pour adresser cette dette technique, une approche réaliste consisterait à se concentrer sur les problèmes "bloquants" et de niveau "élevé", dont la résolution est estimée à environ 80 heures et 40 minutes. Cela améliorerait considérablement la stabilité et réduirait les risques de plantage de l'application.

En synthèse, bien que ShareX bénéficie de sa nature open-source et de sa polyvalence, il doit impérativement améliorer la qualité de son code, son expérience utilisateur et sa documentation. Il est crucial de :

- **Améliorer la qualité du code et les pratiques de développement** : L'implémentation de tests unitaires, l'établissement de conventions de codage et l'utilisation d'outils de surveillance de la qualité du code sont essentielles pour prévenir l'accumulation de dette technique et renforcer la fiabilité et la sécurité.
- **Optimiser l'expérience utilisateur** : Moderniser l'interface, offrir des tutoriels complets pour les nouveaux utilisateurs et des explications claires pour les fonctionnalités avancées rendrait ShareX plus accessible et attractif.
- **Renforcer la communication et la documentation** : Une meilleure gestion du compte X (Twitter) et la création d'une documentation détaillée du code ainsi que de lignes directrices pour les contributeurs favoriseraient une communauté plus engagée et une collaboration plus fluide.
- **Sensibiliser aux risques de confidentialité** : La désactivation par défaut de l'envoi automatique d'images ou une explication détaillée des risques de confidentialité lors de l'installation sont nécessaires pour garantir que les utilisateurs sont pleinement conscients de la destination de leurs données.

En se concentrant sur ces axes d'amélioration, ShareX pourra capitaliser sur ses forces tout en corrigeant ses faiblesses, augmentant ainsi sa valeur pour les utilisateurs actuels et futurs contributeurs.